A guide to ancilla-mediated fixed-point search

Reference: Ancilla-mediated fixed-point quantum search using Grover iterations, by Yash Prabhat, Snigdha Thakur and Ankur Raina. This guide follows the manuscript dated November 7, 2026. The repository includes that PDF at docs/manuscript-2026-11-07.pdf. Equation numbers refer to that version.

What the search does

The data register holds N candidates, of which M are marked by an oracle. With uniform preparation, theta = arcsin(sqrt(M/N)). The normalized state |S> groups the marked candidates and |R> groups the others. The initial data state is cos(theta)|R> + sin(theta)|S>.

Grover's operator G rotates this state by 2 theta per iteration. If M is unknown, the right number of iterations is unknown too. Running too long can reduce the chance of finding a solution.

This method adds an ancilla that flags success. Each round applies a scheduled number of Grover operations, applies F, and measures the ancilla. If the flag is 1, the data register holds a solution and the search stops. If the flag is 0, the data register is left at cos(2 theta)|R> + sin(2 theta)|S>, up to an overall phase. The next round starts from that state.

How F is built

Appendix A factors F into D C_S. The flag operator C_S = H_a (controlled-O) H_a flips the ancilla on marked states. D applies I - 2|u><u| to the data only when the ancilla is 0, where |u> is the prepared uniform state. On ancilla 1, D is the identity.

The sign of the reflection matters because it acts on only one branch. The notebook and theta_emerges.py use this sign and reproduce the four-dimensional matrix in Eq. (13).

For an incoming data angle phi, F produces -cos(phi)|0>(cos(2 theta)|R> + sin(2 theta)|S>) + sin(phi)|1>|S>. Thus the flag succeeds with probability sin(phi)^2. On failure, the normalized data state resets to angle 2 theta. These statements are Eqs. (18)-(19).

The gates do not take theta as an input. The oracle supplies the marked set, and the angle describes the resulting overlap. The circuit structure is the same for different marked sets, while the oracle changes.

Schedule, success and failure

The schedule does not need M

Eq. (22) chooses r = ceil(log2(pi sqrt(N))) checks, indexed j=0,...,r-1. Eq. (23) gives g_0=0 and g_j=2^(j-1)-1 for j>=1. The new Grover operations per round are 0, 0, 1, 3, 7, ... These are not cumulative counts.

The first check starts at theta. Later checks start from the failed branch at 2 theta and amplify to 2^j theta. The conditional success probability is p_j = sin(2^j theta)^2, as in Eq. (25).

The probability of first success at check j is p_j times the probability of failing every earlier check. Eq. (30) gives cumulative success as one minus the product of all failure probabilities through j. Only this accumulated success is monotone; individual p_j values can rise or fall.

What the guarantee means

For at least one solution, the schedule includes the checks needed for the two-check bound in Eqs. (27)-(31). Its exact value is 25/27, about 92.6%. For M=N, the first check succeeds with certainty. For M=0, no check can succeed and every attempt pays the full query budget.

A final "not found" result does not prove that M=0. An attempt can miss an existing solution. The complementary bound is 2/27, about 7.4%, for an ideal completed schedule with M>0. These are probability statements, not guarantees for every finite sample of attempts.

Worked example: N=8, M=1

Mark state |101>. Then theta is about 0.361367 radians, and r=4. The following values use the same schedule as the notebook.

Check j	New G	Queries	Conditional success	Cumulative success
0	0	1	0.125000	0.125000
1	0	2	0.437500	0.507813
2	1	4	0.984375	0.992310
3	3	8	0.061523	0.992783

The final failure probability is 0.007217288. The mean cost, including those failed attempts, is 2.890136719 queries. The maximum is 8 queries. This is a concrete example of a nonempty marked set that can still return "not found".

Count queries consistently

Three cost quantities

One G or F costs one oracle call under the manuscript's convention. Eq. (32) gives the new cost per check; Eq. (33) gives cumulative cost Q_j=2^j. Let w_j be the probability of first success at j, and let J be the last scheduled check.

Eq. (35) gives S_J = sum from j=0 to J of 2^j w_j. This is the successful-event cost sum. The mean cost of all attempts is E_J = S_J + 2^J P(failure). The mean cost conditioned on success is S_J/P(success), when success has nonzero probability.

For the notebook's default N=512, M=1 experiment, S_J=22.073504814, E_J=22.757210882, and the maximum is 64 queries. The README and notebook report E_J. Failed attempts are part of that average.

Two stopping cutoffs

Algorithm 1 fixes J=r-1 from N alone. The manuscript's numerical figure calculation uses the first L for which 2^L theta >= pi/2. L depends on M and is an analytical cutoff, not an implementable unknown-M stopping rule. The research plots show both conventions.

At N=512, M=3, the figure cutoff ends at check 5 and its all-attempt mean is 13.269983331. The fixed schedule ends at check 6 and its all-attempt mean is 14.555071257. The difference comes from the attempts that continue.

The 1.33 epsilon comparison

Epsilon follows the manuscript's Eq. (6): floor(pi/4 sqrt(N/M) - 1/2). It is a small-angle integer benchmark, not the exact optimal Grover count for every marked fraction. Ratios to epsilon are undefined if M=0 or epsilon=0.

On the N=10^10 grid used by the research plots, the mean ratio to epsilon is 1.326455 for all attempts at the figure cutoff and 1.465243 for all attempts in the fixed schedule. The Eq. (35) sum at the figure cutoff has mean ratio 1.257464. These are unweighted grid averages. The approximate 1.33 factor is not a universal constant.

A success budget is not a maximum runtime

Eq. (34)'s pi sqrt(N/M) budget is sufficient to reach the success guarantee. Unsuccessful attempts may continue. The maximum cost of an attempt is 2^(r-1), which depends on N. For N=512, M=511, that maximum is 64 even though pi sqrt(N/M) is about 3.145.

Use and check the repository

Main demonstration

Open notebooks/fixed_point_search.ipynb. Its default ten qubits mean nine data qubits and one ancilla, so N=512. The default marked state is 37. Theory and exact evolution both give success probability 0.9893170927. With 10,000 attempts and seed 12345, the saved sample contains 9,868 successes.

The fast engine updates amplitudes directly. The Qiskit engine applies the gate circuits. Both follow the failed measurement branch and stop successful attempts. The sample is drawn from those computed branch probabilities. It is not a noisy hardware simulation.

Small operator checks

Run python Codes/theta_emerges.py. It constructs F for N=8 with marked sets {5}, {5,6}, and {4,5,6,7}. The first-check flag probabilities are 0.125, 0.25 and 0.5. The script checks unitarity and the normalized conditional states before reporting success. A branch with zero probability has no conditional state to compare.

Run python Codes/verify_updated.py for the bound, schedule and query-cost checks. Run python -m unittest discover -s tests -v for the regression tests, including comparisons with the notebook's gate implementation.

Numerical figures

Codes/FOQA_Complexity_corrected.ipynb uses Codes/make_figures.py for its plots. The script exports the numerical plots associated with manuscript Figs. 3, 5, 6 and 7, plus an extra two-check diagnostic. It does not generate circuit diagrams. Output names use the November 7 figure numbering. Each cost curve identifies its cutoff and whether failed attempts are included.

Notes on the reference

The supplied manuscript is kept unchanged. docs/manuscript_notes.md records the remaining wording qualifications: rounded probability bounds, the interpretation of the guarantee budget, the conservative schedule length, and the statement about the ancilla-1 sector in Appendix A. Successful runs populate |1,S>; the search then stops. They never populate |1,R> in this ideal model.

The query convention assumes a controlled oracle costs one query. Building that oracle, decomposing the gates and handling device noise require their own resource estimates. The examples use uniform state preparation.

This guide's editable source is Codes/companion.md. tools/build_companion.py rebuilds the PDF. The version and formula notes are part of the explanation, so keep them with the plots when sharing results.